

무엇을 만드는가

practice1 — Galaxy Device Issue Triage Console (이슈 분류 웹 페이지)

웹앱 결과

- Galaxy 디바이스 이슈 카드 목록
- P0 / P1 / P2 summary 개수
- Owner Review Needed 개수
- 필터: All / P0 / P1 / P2 / Camera / Battery / Connectivity / Foldable UX / UI / Performance
- P0 카드 강조
- 카드별 priority · requiredTests · owner 표시

Git·문서 증거

```
main_codex_project/  
docs/practice1/  
  backend/report.md  
  frontend/report.md  
  integration/report.md  
  integration/test_report.md  
  integration/approval_draft.json
```

역할마다 증거 파일 경로가 다르다. 그래서 Backend와 Frontend가 동시에 작업해도 report 충돌이 나지 않는다.

이 실습의 진행 도구 — Codex CLI 하나로 끝낸다

사용하는 것

- 터미널(PowerShell)
- git — worktree / branch / commit / merge
- Codex CLI — 각 worktree 폴더에서 `codex` 실행
- Python 표준 라이브러리 — 서버와 검증 스크립트

사용하지 않는 것

- Codex Desktop 앱
- 앱의 "새 작업 트리" 자동 생성 기능
- `~/.codex/worktrees/...` 같은 숨겨진 자동 경로
- 외부 패키지 (`pip install` / `npm install`)

핵심 변경점: worktree는 앱이 만들어주지 않는다. 내가 터미널에서 `git worktree add` 한 줄로 폴더와 branch를 동시에 만든다. 작업 폴더가 눈에 보이는 위치에 생기므로, 어디서 무슨 일이 일어나는지 끝까지 추적할 수 있다.

Starter 구조

```
practice1_mx_issue_console_starter\  
  main_codex_project\  
    server.py  
    index.html  
    src\  
      app.js  
      styles.css  
    data\issues.json          # 실습을 위해 준비된 이슈 로그 데이터  
    tasks\                   # 각 worktree가 독립된 환경에서 작업해야 할 task 문서  
      acceptance.json  
      practice1_main.md  
      practice1_backend.md  
      practice1_frontend.md  
      practice1_integration.md  
    scripts\  
      check_api.py  
      check_dom.py  
      audit.py  
    docs\practice1\          # 각각의 worktree가 작업한 결과물  
      backend\  
      frontend\  
      integration\  
    AGENTS.md  
    README.md
```

빈 폴더는 Git에 저장되지 않을 수 있다. 각 Agent 프롬프트는 필요한 docs 하위 폴더를 직접 만들도록 지시한다.

이슈 데이터 분류 기준 — Backend 규칙 요약

priority

- P0: severity=High + impactScope $\in$ {System, Multi-device}
- P0: 또는 reproRate ≥ 70 + area $\in$ {Camera, Battery, Connectivity, Foldable UX}
- P1: severity=Medium 또는 reproRate ≥ 40 또는 impactScope=App-wide
- P2: 그 외

requiredTests / owner

- 기본: **Smoke**
- P0/P1: **Regression**
- Multi-device: **Device Matrix**
- area별 테스트 추가
- ownerReview / owner는 **acceptance.json** 기준

프롬프트에는 규칙을 길게 복사하지 않는다. Agent가
`tasks/acceptance.json`을 단일 진실 공급원으로 읽게 한다.

준비된 starter 골격

Backend stub

```
def classify_issue(issue):  
    return {  
        "id": issue.get("id"),  
        "priority": "P2",  
        "requiredTests": ["Smoke"],  
        "ownerReview": False,  
        "owner": "",  
        "reasons": ["TODO: not implemented"],  
    }
```

→ 모든 이슈가 P2라서 `check_api` FAIL.

Frontend stub

- 필수 DOM id는 있으나 `#filter-bar` 가 비어 있음
- `#issue-board` 는 placeholder 수준
- `app.js` 는 `/api/issues` 만 호출
- `/api/triage` 호출·렌더·필터 없음
- `.card-p0` 강조 클래스 없음

→ `check_dom` FAIL.

실습 파이프라인 (4단계로 단축)

워크플로우

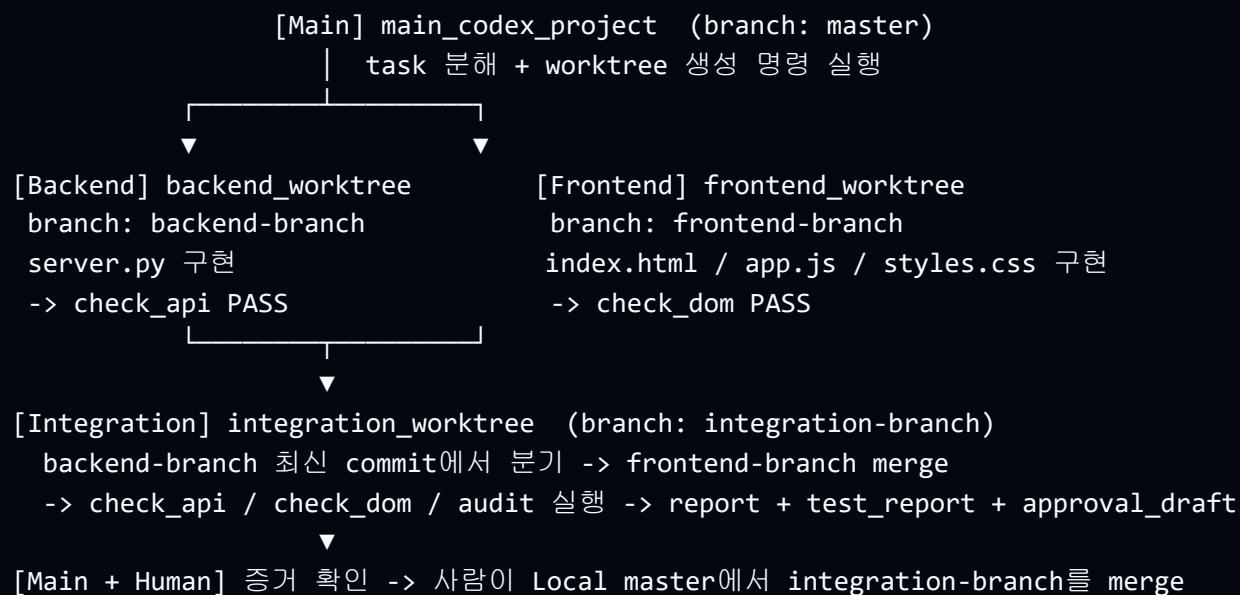
1. Local Main(**master**)에서는 작업 분해와 최종 리뷰만 한다.
2. **master** 에서 **backend_worktree** / **frontend_worktree**를 만들고 각각 **backend-branch** / **frontend-branch**를 붙인다. 두 환경에서 Agent가 **동시에** 구현하고 commit한다.
3. **integration_worktree**를 **backend-branch** 위에 만들고 **integration-branch**에서 **frontend-branch**를 merge한다.
4. 같은 **integration_worktree**에서 이어서 통합 테스트를 돌리고 test_report와 approval_draft까지 남긴다.
5. Human만 Local **master** 에 최종 merge한다.

신뢰 기준

- Agent의 말이 아니라 **changed files**
- 요약이 아니라 **diff**
- 주장보다 **terminal test output**
- 대화보다 **Git branch + commit**
- 외부 폴더보다 **repo 내부 report**

한 줄 요약: 역할을 나누고 → 격리된 작업 트리에서 병렬 구현 → 통합 branch에서 합치고 **바로 검증까지 끝내고** → 사람이 merge한다.

worktree 설계 구조



Backend와 Frontend가 commit되어도 master 파일은 아직 바뀌지 않는다. master가 바뀌는 순간은 사람이 마지막에 merge하는 순간이다.

최종 Git 구조

```
M0 master (starter commit)
|
├─ B1 backend-branch
│   server.py + backend report
|
├─ F1 frontend-branch
│   index.html + src/app.js + src/styles.css + frontend report
|
└─ I1 integration-branch
    = B1에서 분기 -> F1 merge -> 통합 테스트 -> integration report
                                   + test_report + approval_draft
    |
    ▼
    Human이 Local master에서 integration-branch를 merge
```

최종 폴더 구조

```
practice1_mx_issue_console_starter\
├─ main_codex_project\      branch: master
├─ backend_worktree\        branch: backend-branch
├─ frontend_worktree\       branch: frontend-branch
└─ integration_worktree\    branch: integration-branch
```

Starting branch & Role branch

어느 구현단계에서 시작하는가 & 어떤 역할과 목적을 가진 branch인가

역할	worktree 폴더	Starting branch	Role branch	이후 동작
Main	main_codex_project	—	master	분해 · 리뷰 · 최종 merge
Backend	backend_worktree	master	backend-branch	API 구현
Frontend	frontend_worktree	master	frontend-branch	UI 구현
Integration	integration_worktree	backend-branch	integration-branch	frontend-branch merge + 통합 테스트

worktree는 별도의 독립된 개발 환경이다. 해당 환경에서 코드 작업 및 변경 사항을 버전별, 단계별로 관리하려면 작업에 따라 별도 branch로 추적해야 한다. git
worktree add ... -b ... 는 폴더와 branch를 한 번에 만든다.

역할(Role branch)별 코드 구현 및 파일 작업 범위

역할	수정 허용	수정 금지
Backend	<code>server.py</code> , <code>docs/practice1/backend/report.md</code>	Frontend 파일, data, scripts, tasks
Frontend	<code>index.html</code> , <code>src/app.js</code> , <code>src/styles.css</code> , <code>docs/practice1/frontend/report.md</code>	Backend 파일, data, scripts, tasks
Integration	merge commit, <code>docs/practice1/integration/</code> 아래 문서 4종	구현 파일 직접 수정
Main	없음	add, commit, merge, push
Human	Local master에서 최종 merge	AI 말만 믿고 승인

Integration이 구현 파일을 직접 고치면 역할 추적이 깨진다. 테스트가 실패하면 담당 역할 branch로 돌려보낸다.

준비 ① · 저장소 초기화

위치: `main_codex_project` · **Human** · 하는 일: starter commit 생성 · 증거: starter commit

starter kit 경로 꼭 확인하고, 터미널에서 아래를 진행

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\main_codex_project"

git init -b master
git add .
git commit -m "chore(practice1): import starter"

git log -1 --oneline --decorate
```

구버전 Git에서 `git init -b master`가 실패하면 `git init` 후 `git branch -m master`를 실행한다.

압축 해제한 스타터킷 폴더에서 반드시 git 저장소를 먼저 만들어야 `git worktree add`가 동작한다. `worktree`는 "이미 존재하는 저장소"에서만 뺄 수 있다.

준비 ② · 초기 FAIL 확인

위치: git 추적 중인 `main_codex_project` · **Human** · 하는 일: 빈 곳 확인 · 증거: 콘솔 출력

```
py scripts/check_api.py
py scripts/check_dom.py
py scripts/audit.py
```

정상적인 초기 실패 출력 예시

```
[check-api] FAIL - 6 failures
[check-dom] FAIL - 3 failures
[audit] api=FAIL dom=FAIL
-> implementationRisk=HIGH
```

이 FAIL은 의도된 것이다.

Backend/Frontend가 채워야 할 목표를 보여주는 출발점이다.

현재 상태 확인 명령

위치: 각 작업 경로의 터미널 · **Human** · 하는 일: 현재 경로/branch 확인 · 증거 확인: 콘솔 출력

```
git branch --show-current    # master인가, 역할 branch인가
git status --short           # 현 branch가 clean인가, 변경 파일이 있는가
git worktree list             # 이 저장소에 연결된 작업 트리 목록
```

worktree를 만들기 전 `git worktree list` 출력

모든 `git worktree add` 명령은 `main_codex_project` 안에서 실행한다. worktree는 저장소를 기준으로 만들어지기 때문이다.

git worktree add 명령어 해부

```
git worktree add ..\backend_worktree -b backend-branch master
#
#
#
```

① 만들 폴더 경로

② 새로 만들 branch

③ 출발점 (시작 branch)

조각	의미	이번 실습 값
..\backend_worktree	작업 트리를 만들 폴더 경로. .. 는 main_codex_project 의 상위 폴더	스타터킷 루트 바로 아래에 생성
-b backend-branch	새 branch를 동시에 생성하고 그 폴더에 checkout	backend-branch
master	새 branch가 어느 commit에서 출발하는지	master (Backend/Frontend), backend-branch (Integration)

-b가 핵심이다. 이 옵션 하나로 폴더 생성 + branch 생성 + checkout이 한 번에 끝난다. 별도로 `git branch`를 치거나, 앱 화면에서 "브랜치 생성"을 누를 필요가 전혀 없다.

단계 1 · Backend / Frontend worktree 생성

위치: `main_codex_project` 터미널 · **Human** · 하는 일: 병렬 작업장 2개 생성 · 증거: 폴더 2개 + branch 2개

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\main_codex_project"
```

```
# 폴더와 branch를 한 번에 생성한다 (둘 다 master에서 출발)
```

```
git worktree add ../backend_worktree -b backend-branch master
```

```
git worktree add ../frontend_worktree -b frontend-branch master
```

```
git worktree list
```

정상 출력

```
.../main_codex_project    a1b2c3d [master]
.../backend_worktree      a1b2c3d [backend-branch]
.../frontend_worktree     a1b2c3d [frontend-branch]
```

세 폴더 모두 같은 commit(a1b2c3d)에서 출발했다. 여기서부터 갈라진다.

Frontend도 반드시 `master` 에서 출발한다. Backend와 Frontend의 작업 범위는 완전히 독립이다. `backend-branch` 위에서 만들면 병렬 구조가 깨진다.

단계 2 · 각 worktree에서 Codex CLI 실행

위치: 새 터미널 2개 · **Backend** **Frontend** · 하는 일: 독립 세션 2개 기동 · 증거: 각 세션의 작업 경로

터미널 A — Backend

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\backend_worktree"

git branch --show-current    # backend-branch
codex
```

터미널 B — Frontend

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\frontend_worktree"

git branch --show-current    # frontend-branch
codex
```

터미널을 두 개 열어 두는 것이 이 실습의 병렬성이다. 각 Codex CLI 세션은 자기가 실행된 폴더만 본다. 그래서 Backend 세션은 frontend_worktree의 파일을 아예 인식하지 못한다. 이것이 격리다.

Codex CLI가 파일 수정·명령 실행 승인을 묻으면 허용한다. 한 번에 실행하려면 `codex exec "<프롬프트>"`도 가능하다.

단계 3 · Backend 구현 프롬프트

위치: `backend_worktree` 의 Codex CLI · **Backend** · 하는 일: API 규칙 구현 · 증거: `backend commit/report`

commit까지 진행한다!

너는 Backend Agent다.

현재 작업 폴더는 `backend_worktree`이고, branch는 `backend-branch`다.

읽기:

- `AGENTS.md`
- `tasks/practice1_backend.md`
- `tasks/acceptance.json`

수정 허용:

- `server.py`
- `docs/practice1/backend/report.md`

수행:

1. `docs/practice1/backend` 디렉터리가 없으면 생성하라.
2. `acceptance.json`에 따라 `classify_issue`를 구현하라.
3. `py scripts/check_api.py`를 PASS시켜라.
4. `report.md`에 branch, 변경 파일, 구현 내용, 테스트 결과, 남은 리스크를 기록하라.
5. 허용 파일만 stage하고 commit하라.

commit message: `feat(practice1): implement issue triage backend`

`git worktree add/remove`, branch 변경, `merge`, `rebase`, `push`는 금지한다.

마지막에 `commit hash`, 변경 파일, 테스트 결과를 보고하라.

단계 4 · Frontend 구현 프롬프트

위치: `frontend_worktree`의 Codex CLI · **Frontend** · 하는 일: 대시보드 구현 · 증거: frontend commit/report

commit까지 진행한다!

너는 Frontend Agent다.

현재 작업 폴더는 `frontend_worktree`이고, branch는 `frontend-branch`다.

읽기:

- `AGENTS.md`
- `tasks/practice1_frontend.md`
- `tasks/acceptance.json`

수정 허용:

- `index.html`
- `src/app.js`
- `src/styles.css`
- `docs/practice1/frontend/report.md`

수행:

1. `docs/practice1/frontend` 디렉터리가 없으면 생성하라.
2. UI, 필터, `/api/issues`, `/api/triage` 호출을 구현하라.
3. `py scripts/check_dom.py`를 PASS시켜라.
4. `report.md`에 branch, 변경 파일, 구현 UI, 테스트 결과, 남은 리스크를 기록하라.
5. 허용 파일만 stage하고 commit하라.

commit message: `feat(practice1): implement issue triage dashboard`

`git worktree add/remove`, branch 변경, `merge`, `rebase`, `push`는 금지한다.

마지막에 `commit hash`, 변경 파일, 테스트 결과를 보고하라.

백업 경로 · 내가 직접 commit하기

위치: 해당 worktree 폴더의 터미널 · **Human** · 하는 일: Agent가 commit을 안 했을 때 직접 마무리 · 증거: commit hash

backend_worktree에서

```
git status --short

git add server.py docs/practice1/backend/report.md
git commit -m "feat(practice1): implement issue triage backend"

git log -1 --oneline --decorate
```

frontend_worktree에서

```
git status --short

git add index.html src/app.js src/styles.css `
      docs/practice1/frontend/report.md
git commit -m "feat(practice1): implement issue triage dashboard"

git log -1 --oneline --decorate
```

프롬프트에서 commit까지 지시했지만, Agent가 승인 거부·중단·오류로 commit을 건너뛸 수 있다. `git status --short`에 파일이 남아 있으면 위 명령으로 직접 마무리한다. 이때도 허용된 파일만 stage한다. `git add .`는 쓰지 않는다.

Backend / Frontend 결과 확인

위치: 각 worktree 터미널 · **Human** · 하는 일: commit/PASS 확인 · 증거: Git log

backend_worktree

```
git branch --show-current    # backend-branch
git status --short           # 출력 없음 = clean
py scripts/check_api.py      # PASS
git log -1 --oneline --decorate
git show --name-status --oneline HEAD
```

frontend_worktree

```
git branch --show-current    # frontend-branch
git status --short           # 출력 없음 = clean
py scripts/check_dom.py      # PASS
git log -1 --oneline --decorate
git show --name-status --oneline HEAD
```

여기까지 완료되면 `backend-branch`와 `frontend-branch`가 각각 자기 commit을 갖는다. Local master에는 아직 아무것도 반영되지 않았다.

`git show --name-status`로 변경 파일 목록이 역할 범위를 벗어나지 않았는지 반드시 눈으로 확인한다. 여기서 범위 위반을 잡아야 Integration에서 충돌이 안 난다.

단계 5 · Integration worktree 생성

위치: `main_codex_project` 터미널 · **Human** · 하는 일: 통합 작업장 생성 · 증거: integration-branch 생성

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\main_codex_project"

# 출발점이 master가 아니라 backend-branch다
git worktree add ..\integration_worktree -b integration-branch backend-branch

git worktree list
```

시작 branch는 `master`가 아니라 백엔드 구현이 완료된 `backend-branch`다!! master는 아직 stub 상태라서, master에서 분기하면 합칠 대상이 없다.

반드시 Backend commit 이후에 생성

`backend-branch`의 최신 commit을 출발점으로 잡기 때문에, Backend가 commit하기 전에 만들면 stub 코드를 들고 시작한다.

Frontend는 merge로 들어온다

Frontend 구현은 폴더 복사가 아니라 `git merge frontend-branch`로 합쳐진다. 그래서 Frontend도 commit이 끝나 있어야 한다.

단계 6 · Integration 프롬프트 (merge + 통합 테스트 + 승인 초안)

위치: `integration_worktree` 의 Codex CLI · **Integration** · 하는 일: merge와 검증을 한 자리에서 · 증거: merge commit + 문서 commit

너는 Integration Agent다.

현재 작업 폴더는 `integration_worktree`이고, branch는 `integration-branch`다.

읽기: `AGENTS.md`, `tasks/practice1_integration.md`, `tasks/acceptance.json`

수행:

1. HEAD가 `backend-branch` 최신 commit과 같은지, `working tree`가 `clean`한지 확인하라.
2. `frontend-branch`를 `--no-ff`로 merge하라. 충돌 시 임의 해결하지 말고 중단하고 보고하라.
3. `py scripts/check_api.py`, `py scripts/check_dom.py`, `py scripts/audit.py`를 실행하라.
4. 구현 파일(`server.py`, `index.html`, `src/`)은 절대 직접 수정하지 마라.
5. `docs/practice1/integration` 아래에만 아래 3개 문서를 작성하라.
 - `report.md` : merge 대상, 충돌 유무, 변경 파일 범위
 - `test_report.md` : 실제 PASS/FAIL, `implementationRisk`, 실패 원인, 남은 리스크
 - `approval_draft.json` : `status`는 항상 `"pending"`,
전부 통과면 `recommendation "approve_candidate"`, 아니면 `"hold"`
6. 문서 3개만 stage해서 merge commit과 별도로 commit하라.
commit message: `docs(practice1): record integration and test evidence`

`master merge`와 `push`는 금지한다. 마지막에 `merge commit hash`와 3개 스크립트 결과를 보고하라.

FAIL이나 충돌은 누군가 역할 범위 밖 파일을 건드렸거나, 구현이 덜 되었다는 신호다. Integration이 급히 고치지 말고 어느 역할이 담당하는 파일인지 확인해서 그 branch로 되돌린다.

백업 경로 · 내가 직접 merge하고 commit하기

위치: `integration_worktree` 터미널 · **Human** · 하는 일: 직접 통합 · 증거: merge commit + 문서 commit

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\integration_worktree"

git branch --show-current      # integration-branch
git status --short             # clean이어야 한다

# (1) Frontend 구현을 병합한다. --no-ff로 merge 이력을 남긴다
git merge --no-ff frontend-branch -m "merge(practice1): merge frontend-branch into integration-branch"

# (2) 통합본 전체 검증
py scripts/check_api.py
py scripts/check_dom.py
py scripts/audit.py

# (3) 증거 문서만 stage해서 별도 commit
git add docs/practice1/integration/
git commit -m "docs(practice1): record integration and test evidence"

git log --oneline --graph -5
```

충돌이 나면 `git merge --abort` 로 되돌린 뒤, 충돌 파일의 담당 역할 branch에서 먼저 정리한다. Integration에서 손으로 해결하면 누가 쓴 코드인지 추적이 끊긴다.

Integration 증거 확인

위치: `integration_worktree` 터미널 · **Human** · 하는 일: 증거 확인 · 증거: report / test_report / approval

```
git status --short          # 출력 없음 = commit 완료, clean
git log --oneline --graph -5 # merge commit + 문서 commit이 보인다
```

Get-Content docs/practice1/integration/test_report.md

Get-Content docs/practice1/integration/approval_draft.json

정상 기준

- check_api PASS
- check_dom PASS
- audit implementationRisk = LOW
- status = **pending**
- recommendation = **approve_candidate**

실패 시

- test_report에서 어느 check가 실패했는지 확인
- Backend / Frontend 중 담당 branch로 되돌려 수정 · 재commit
- Integration에서 해당 branch를 다시 merge하고 재검증

approval_draft의 status는 어떤 경우에도 **pending**이다. Agent가 스스로 approved로 바꿔 놓았다면 그것 자체가 규칙 위반이므로 반려한다.

단계 7 · Local Main Review — 범위 diff

위치: Local Main(master) 터미널 · **Main** + **Human** · 하는 일: branch 증거 종합 · 증거: review 판단

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\main_codex_project"

git branch --show-current      # master

git diff --name-status master..backend-branch
git diff --name-status master..frontend-branch
git diff --name-status backend-branch..integration-branch
```

이 명령은 Human merge **전에** 실행한다. merge 후에는 master가 변경을 이미 포함하므로 비교 의미가 흐려진다.

내용 확인

```
git diff master..backend-branch -- server.py
git diff master..frontend-branch -- index.html src/app.js src/styles.css

git show integration-branch:docs/practice1/integration/test_report.md
git show integration-branch:docs/practice1/integration/approval_draft.json
```

`git show branch:path` 는 checkout 없이 해당 branch에 commit된 report를 읽는다. 아직 master에 병합되지 않은 문서도 확인할 수 있다.

단계 8 · Main Reviewer 프롬프트

위치: `main_codex_project` 의 Codex CLI · **Main** · 하는 일: 증거 기반 판단 · 증거: approve/hold/reject 제안

너는 Local master의 Main Reviewer다.

수정·commit·merge·push하지 마라. 읽고 판단만 하라.

직접 `git diff`, `git show`, `git log` 명령을 실행해 다음 증거를 읽어라.

- `master..backend-branch`, `master..frontend-branch` 의 diff
- `backend-branch..integration-branch` 의 diff
- `backend / frontend / integration report`
- integration의 `test_report.md`와 `approval_draft.json`
- `git log --oneline --graph --all`

확인:

- 역할별 파일 범위 위반 여부
- `check_api / check_dom` 결과
- `implementationRisk`
- `approval status`가 `pending`인지
- Integration이 구현 코드를 직접 수정하지 않았는지

`approve / hold / reject` 중 하나를 이유와 함께 제안하라.

최종 결정과 `merge`는 사람이 한다.

 Main Reviewer 세션도 `git status --short` 가 비어 있어야 정상이다. 읽고 판단만 한다.

단계 9 · Human Merge

위치: Local Main(master) 터미널 · **Human** · 하는 일: 최종 승인 후 merge · 증거: master merge commit

```
Set-Location "$HOME\Desktop\practice1_mx_issue_console_starter\main_codex_project"
```

```
git switch master  
git status --short
```

```
git merge --no-ff integration-branch `   
  -m "merge(practice1): complete parallel issue console implementation"
```

```
py scripts/check_api.py  
py scripts/check_dom.py  
py scripts/audit.py
```

```
git log --oneline --graph --all
```

integration-branch 는 **backend-branch** 위에서 만들어졌고 **frontend-branch** 를 merge했으므로, 이 **branch** 하나만 merge하면 Backend·Frontend·Integration 구현과 증거가 함께 master로 들어온다.

commit ≠ master 반영

Agent commit 시점

- 해당 역할 branch에는 반영된다.
- Local에서도 조회 가능하다.
- 하지만 master 파일은 아직 바뀌지 않는다.

```
git show backend-branch:server.py
```

Human merge 시점

- 이때 Local master 파일이 최종본으로 바뀐다.
- 브라우저로 열 앱도 이 시점부터 완성본이다.

master를 바꾸는 단 하나의 행위는 사람이 실행하는 최종 merge다.

단계 10 · 최종 웹앱 확인

위치: Local Main(master) 터미널 · **Human** · 하는 일: 완성본 실행 · 증거: 브라우저 화면

```
py server.py
```

브라우저: `http://127.0.0.1:8000`

보여야 할 것

- 이슈 카드 목록
- P0/P1/P2 summary
- Owner Review 개수
- 필터 버튼 동작
- P0 카드 강조
- priority·requiredTests·owner 표시

데이터 변경 데모 후 복구

`data/issues.json`에 이슈를 추가해 새로그침 테스트를 했다면:

```
git restore data/issues.json  
git status --short
```

단계 11 · worktree 정리

위치: Local Main(master) 터미널 · **Human** · 하는 일: 작업 트리 회수 · 증거: worktree list

```
cd "$HOME\Desktop\practice1_mx_issue_console_starter\main_codex_project"
```

```
# 각 worktree 폴더가 clean한지 먼저 확인한 뒤 제거한다
```

```
git worktree remove ..\backend_worktree
```

```
git worktree remove ..\frontend_worktree
```

```
git worktree remove ..\integration_worktree
```

```
git worktree list      # main_codex_project 하나만 남는다
```

```
git branch             # branch 이력은 그대로 남아 있다
```

폴더가 사라져도 작업은 남는다. worktree는 "작업장"일 뿐이고, 실제 결과물은 이미 branch와 commit에 저장되어 있다. `git log`, `git show backend-branch:server.py`로 언제든지 조회할 수 있다.

커밋되지 않은 변경이 남아 있으면 `git worktree remove`가 거부된다. 그때는 해당 폴더에서 commit을 마무리하거나 변경을 버린 뒤 다시 실행한다.

실패 복구 절차

Backend / Frontend 실패

- 해당 worktree 폴더로 돌아간다.
- 수정·재테스트·새 commit을 만든다.
- Integration worktree에서 해당 branch를 **다시 merge**한다.

```
# integration_worktree에서  
git merge --no-ff frontend-branch  
py scripts/audit.py
```

Integration 실패

- Integration은 구현 파일을 직접 수정하지 않는다.
- 담당 역할 branch로 반환한다.
- merge를 되돌리려면:

```
git merge --abort      # merge 진행 중일 때  
git reset --hard HEAD~1  # merge commit을 취소할 때
```

같은 branch를 여러 worktree에서 동시에 checkout할 수 없다(Git이 거부한다). 처음부터 다시 하려면 `integration-branch-r2` 처럼 새 **branch 이름**과 새 **폴더 이름**으로 분리한다.

예: `git worktree add ../integration_worktree_r2 -b integration-branch-r2 backend-branch`

핵심 해석

병렬 worktree의 본질

- 작업 트리는 branch 기준의 **독립 작업장**이다.
- `git worktree add ... -b ...` 한 줄이 폴더 ·branch·checkout을 동시에 처리한다.
- Backend와 Frontend는 master에서 동시에 분기한다.
- Integration은 두 branch를 합치고 **그 자리에서 검증까지 끝낸다.**
- 앱 기능에 의존하지 않으므로, 어떤 터미널 환경에서도 재현된다.

증거와 승인

- 구현 전달은 **branch + commit.**
- 판단 전달은 **role report.**
- 검증 전달은 **test_report + approval draft.**
- approval draft는 항상 **pending.**
- master merge는 **사람만** 한다.

Main이 나누고 → Backend·Frontend가 병렬 구현·commit → Integration이 merge하고 검증까지 마무리 → Human이 merge한다.